

compressed files created by another user on another system. But FSlint finds and reports such problems. Figure 7 shows the results of the operation.

In this example, the FSlint utility reports a file with a bad ID because there is no user or group with ID 1001.

Empty directories

Empty directories clutter a file system, and make it difficult for the average user to find information quickly and efficiently. FSlint can find and clean these annoying empty directories (Figure-8 shows the result of the operation).

Non-stripped binaries

Non-stripped binaries contain extra debugging symbols and tend to be larger in size. These extra debugging symbols are needed while debugging binary files with debuggers like GDB. But often, this extra debugging information is not needed, and a considerable amount of drive space can be freed by stripping the binaries. FSlint finds such non-stripped binaries (Figure-9 shows the result of the operation).

In this example, the *hello* binary was purposefully compiled to contain debugging information.

With this tool, when the *Search \$PATH* check box is enabled, FSlint searches the system path for non-stripped binaries. This tool provides a *Clean* button, which removes extra debugging symbols from the binary.

Redundant white spaces

FSlint can check text files for a number of white space issues like unnecessary tabs and spaces. This feature is very useful for programmers and writers who need to be aware of the white space within their files. The average user may never need to use this feature, but knowing about this tool could be beneficial.

This tool also provides the check box for *Bad indenting* for *indent width* which checks text files to ensure that the indenting width is uniform. FSlint can also check for a white space at the end of a line. The *Clean* button will attempt to fix the white space issues in the selected files (Figure-10 shows the result of the operation).

To try this out, let us create a file with a trailing white space, which will look like what follows:

```
[narendra]$ cat -e hello.c
#include <stdio.h>  $
$
int main(void)  $
{
    $
    printf("Hello, World !!!\n");  $
$
    return 0;  $
}  $
```

Now, remove the trailing white space using FSlint's clean option. Our modified file will now look like what's shown below:

```
[narendra]$ cat -e hello.c
#include <stdio.h>
$
int main(void)
{
    printf("Hello, World!!!\n");
$
    return 0;
}$
```

In the above examples, the dollar (\$) symbol implies the end of the line.

FSlint is an extremely useful tool that can be used regularly.

These simple yet powerful utilities make GNU/Linux more interesting. You can always learn more about FSlint by digging into it yourself. **END** 

By: Narendra Kangralkar

The author is a FOSS enthusiast and loves exploring anything related to open source. He can be contacted at narendrakangralkar@gmail.com.



2 Simple Ways To Stay Up-to-date With India's Electronics Industry

Want Latest News?
Check out
EFYTimes.com



Want Industry Trends?
Check out
Electronics2b.com



For any queries, please contact our team at efyeng@efyindia.com OR +91-11-26810601